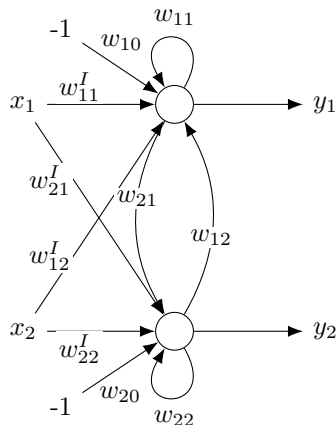


1. Arquitectura y pasada hacia adelante

Para la arquitectura neuronal



la salida en un instante dado t es

$$\mathbf{y}_t = \phi(W^I \mathbf{x}_t + W \mathbf{y}_{t-1}),$$

donde W^I y W son las matrices de pesos y $\phi(\cdot)$ es la función de activación. Para los siguientes desarrollos nos será útil expresar la ecuación anterior en forma escalar, para cada neurona j y cada entrada i ,

$$y_{t,j} = \phi(v_{t,j}) = \phi\left(\sum_i w_{ji}^I x_{t,i} + \sum_i w_{ji} y_{t-1,i}\right),$$

con función de activación sigmoidea simétrica,

$$\phi(v) = \frac{2}{1 + e^{-v}} - 1.$$

El Algoritmo 1 muestra la pasada hacia adelante de la red para una secuencia X , de longitud T .

Algoritmo 1 RNN pasada hacia adelante (forward)

- 1: **Entradas:** Secuencia X (en one-hot), estado inicial $\mathbf{y}_{-1}$
 - 2: **Salida:** Predicciones $\mathbf{y}$
 - 3: **for** $t = 0$ **to** $T - 1$ **do**
 - 4: $\mathbf{y}_t \leftarrow \phi(W^I \mathbf{x}_t + W \mathbf{y}_{t-1})$
 - 5: **end for**
 - 6: **return** $\mathbf{y}$
-

En este algoritmo se han supuesto entradas y salidas categóricas, como por ejemplo secuencias de palabras, pero la extensión al caso de variables continuas es directa eliminando la codificación one-hot. Por otro lado, si bien en la figura se muestra una entrada

con dimensión igual a la cantidad de neuronas, el vector $\mathbf{x}_t$ puede tener cualquier dimensión y lo único que cambiaría es la dimensión de la matriz de pesos W^I . En este mismo sentido también vale aclarar que en esta red neuronal sencilla no se ha incorporado una capa de salida, por lo que la dimensión de la salida será igual a la cantidad de neuronas.

A continuación veremos cómo entrar esta red neuronal. Una vez entrenada, se puede utilizar para generar secuencias a partir de una entrada inicial. El proceso es similar a la pasada hacia adelante, pero en cada paso se utiliza la salida generada previamente como entrada para el siguiente paso. El Algoritmo 2 muestra cómo se puede generar una secuencia de longitud T a partir de una entrada inicial $\mathbf{x}_0$.

Algoritmo 2 Generación de una secuencia

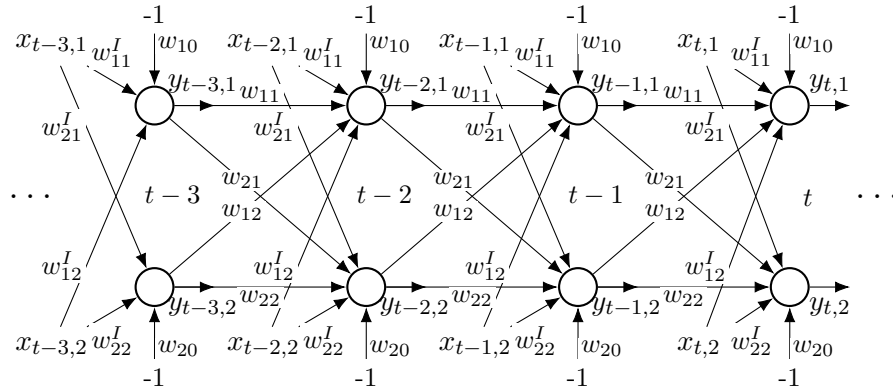
```

1: Entradas: Inicio  $\mathbf{x}_0$  (en one-hot), longitud deseada  $T$ 
2: Salida: Secuencia generada  $Y$ 
3:  $\mathbf{y} \leftarrow \mathbf{0}, \mathbf{x} \leftarrow \mathbf{x}_0$ 
4:  $Y \leftarrow [\mathbf{x}_0]$ 
5: for  $t = 1$  to  $T - 1$  do
6:    $\mathbf{y} \leftarrow \text{forward}(\mathbf{x}, \mathbf{y})$ 
7:    $\mathbf{x} \leftarrow \text{OneHot}(\mathbf{y})$ 
8:    $Y \leftarrow Y + [\mathbf{x}]$ 
9: end for
10: return  $Y$ 

```

2. Retropropagación a través del tiempo

Para entrenar esta red neuronal vamos a considerar la arquitectura desenrollada a lo largo del tiempo



para la cual el error total es

$$E = \sum_{t=1}^T E_t = \frac{1}{2} \sum_{t=1}^T \sum_{k=1}^N e_{t,k}^2 = \frac{1}{2} \sum_{t=1}^T \sum_{k=1}^N (y_{t,k} - d_{t,k})^2.$$

2.1. Pesos recurrentes

Tomemos primero el caso de los pesos recurrentes W . Para el error total tenemos que

$$\frac{\partial E}{\partial w_{ji}} = \frac{\partial E_0}{\partial w_{ji}} + \frac{\partial E_1}{\partial w_{ji}} + \dots + \frac{\partial E_T}{\partial w_{ji}}.$$

Consideremos el primer paso, para $t = 0$, a partir de un estado inicial $\mathbf{y}_{-1}$. La red se desenrolla en solo una capa. Tenemos que $E_0 = \frac{1}{2} \sum_{k=1}^N (y_{0,k} - d_{0,k})^2$, depende de los pesos W que actúan sobre $\mathbf{y}_{-1}$, por lo que el gradiente se calcula como

$$\frac{\partial E_0}{\partial w_{ji}} = \frac{\partial E_0}{\partial y_{0,j}} \frac{\partial y_{0,j}}{\partial v_{0,j}} \frac{\partial v_{0,j}}{\partial w_{ji}}.$$

En el primer factor tenemos que

$$\frac{\partial E_0}{\partial y_{0,j}} = \frac{\partial}{\partial y_{0,j}} \frac{1}{2} \sum_{k=1}^N e_{0,k}^2 = \sum_{k=1}^N e_{0,k} \frac{\partial e_{0,k}}{\partial y_{0,j}} = e_{0,j},$$

ya que $\partial e_{0,k} / \partial y_{0,j} = 1$ solo cuando $k = j$. Para el segundo factor, sabemos que la función de activación sigmoidea simétrica tiene $\partial y_{0,j} / \partial v_{0,j} = (1 - y_{0,j})(1 + y_{0,j})$. Y para la última derivada, ya con respecto al peso w_{ji} , tenemos

$$\frac{\partial v_{0,j}}{\partial w_{ji}} = \frac{\partial}{\partial w_{ji}} \sum_{\ell} w_{j\ell} y_{-1,\ell} = y_{-1,i}.$$

De esta forma nos queda

$$\frac{\partial E_0}{\partial w_{ji}} = (y_{0,j} - d_{0,j})(1 - y_{0,j})(1 + y_{0,j}) y_{-1,i}.$$

Si llamamos

$$\delta_{0,j} \triangleq \frac{\partial E_0}{\partial y_{0,j}} \frac{\partial y_{0,j}}{\partial v_{0,j}} = (y_{0,j} - d_{0,j})(1 - y_{0,j})(1 + y_{0,j}),$$

podemos escribir de forma simplificada

$$\frac{\partial E_0}{\partial w_{ji}} = \delta_{0,j} y_{-1,i}.$$

Es interesante notar que $(1 - y_{0,j})(1 + y_{0,j})$, que se da solo en el caso de la función de activación sigmoidea simétrica, es un factor que limita al $\delta_{0,j}$, evitando que los gradientes crezcan demasiado cuando hay muchas capas desenrolladas. Para otras funciones de activación suele ser necesario incorporar mecanismos adicionales para evitar el problema de la explosión de gradientes, como la normalización o el recorte (clipping) de gradientes.

Ahora avanzamos desenrollando la red hasta $t = 1$. En este caso, la red neuronal pasaría a tener dos capas. El error E_1 depende de dos aportes que involucran a w_{ji} : uno directo, a través del w_{ji} que actúa en $t = 1$, y otro indirecto, a través del w_{ji} que actúa en $t = 0$. En el siguiente gráfico se esquematizan los dos aportes de forma simplificada.

Entonces, para el segundo término tenemos que

$$\begin{aligned}
\frac{\partial E_1}{\partial w_{0,ji}} &= \frac{\partial E_1}{\partial y_{0,j}} \frac{\partial y_{0,j}}{\partial v_{0,j}} \frac{\partial v_{0,j}}{\partial w_{0,ji}} \\
&= \frac{\partial}{\partial y_{0,j}} \frac{1}{2} \sum_{k=1}^N e_{1,k}^2 \cdot (1 - y_{0,i})(1 + y_{0,i}) \cdot y_{-1,i} \\
&= \sum_{k=1}^N e_{1,k} \frac{\partial e_{1,k}}{\partial y_{0,j}} \cdot (1 - y_{0,i})(1 + y_{0,i}) y_{-1,i} \\
&= \sum_{k=1}^N e_{1,k} \frac{\partial e_{1,k}}{\partial y_{1,k}} \frac{\partial y_{1,k}}{\partial v_{1,k}} \frac{\partial v_{1,k}}{\partial y_{0,j}} \cdot (1 - y_{0,i})(1 + y_{0,i}) y_{-1,i} \\
&= \sum_k e_{1,k} \cdot 1 \cdot (1 - y_{1,k})(1 + y_{1,k}) \cdot w_{1,jk} \cdot (1 - y_{0,i})(1 + y_{0,i}) y_{-1,i} \\
&= \left(\sum_k \delta_{1,k} w_{1,jk} \right) (1 - y_{0,i})(1 + y_{0,i}) y_{-1,i} \\
&= \delta_{0,j} y_{-1,i}.
\end{aligned}$$

En esta ecuación hay que tener en cuenta que el $\delta_{1,k}$ que aparece dentro de la última sumatoria es el mismo $\delta_{1,j}$ que teníamos en el [aporte directo](#) del primer término, pero ahora con k en lugar de j porque se refiere a las neuronas de salida en $t = 1$. Más aún, la retropropagación del $\delta_{1,k}$ hacia atrás, a través de las conexiones recurrentes con $w_{1,jk}$, es en realidad el $\delta_{0,j}$. Pero hay que tener mucho cuidado con la notación, porque este $\delta_{0,j}$ **no es** el mismo que el que se obtuvo a partir de E_0 , cuando la red se había desenrollado solamente hasta $t = 0$. Ahora, este $\delta_{0,j}$ se obtiene a partir de la retropropagación del error desde $t = 1$ hacia atrás, es decir, viene del error E_1 (no del E_0). Para diferenciarlos mejor se podría haber usado $\delta_{0,j}^{(E_0)}$ y $\delta_{0,j}^{(E_1)}$, pero preferimos no sobrecargar tanto la notación.

Finalmente, sumando los aportes directo e indirecto tenemos

$$\frac{\partial E_1}{\partial w_{ji}} = \delta_{1,j} y_{0,i} + \delta_{0,j} y_{-1,i} = \sum_{\tau=1}^0 \delta_{\tau,j} y_{\tau-1,i},$$

que podemos generalizar para cualquier t mediante

$$\frac{\partial E_t}{\partial w_{ji}} = \sum_{\tau=t}^0 \delta_{\tau,j} y_{\tau-1,i},$$

con los $\delta_{\tau,j}$ definidos recursivamente como

$$\delta_{\tau,j} = \begin{cases} (y_{\tau,j} - d_{\tau,j}) (1 - y_{\tau,j})(1 + y_{\tau,j}), & \text{si } \tau = t, \\ \left(\sum_k w_{jk} \delta_{\tau+1,k} \right) (1 - y_{\tau,j})(1 + y_{\tau,j}), & \text{si } \tau < t. \end{cases}$$

A partir de esta ecuación generalizada, ya podemos calcular el gradiente global

$$\frac{\partial E}{\partial w_{ji}} = \sum_{t=1}^T \sum_{\tau=t}^0 \delta_{\tau,j} y_{\tau-1,i},$$

y actualizar los pesos mediante

$$w_{ji} \leftarrow w_{ji} - \eta \frac{\partial E}{\partial w_{ji}}.$$

2.2. Pesos de entrada

Ahora tomemos el caso de W^I . El desarrollo es muy similar al caso anterior. Para el error total tenemos que

$$\frac{\partial E}{\partial w_{ji}^I} = \frac{\partial E_0}{\partial w_{ji}^I} + \frac{\partial E_1}{\partial w_{ji}^I} + \dots + \frac{\partial E_T}{\partial w_{ji}^I}.$$

Consideramos primero el desenrollado hasta $t = 0$, donde el error $E_0 = \frac{1}{2} \sum_{k=1}^N e_{0,k}^2$ solo depende de los pesos W^I actuando sobre la entrada en $t = 0$. Por lo tanto, el gradiente es

$$\frac{\partial E_0}{\partial w_{ji}^I} = \frac{\partial E_0}{\partial y_{0,j}} \frac{\partial y_{0,j}}{\partial v_{0,j}} \frac{\partial v_{0,j}}{\partial w_{ji}^I}.$$

Como antes, el primer factor es

$$\frac{\partial E_0}{\partial y_{0,j}} = \frac{\partial}{\partial y_{0,j}} \frac{1}{2} \sum_{k=1}^N e_{0,k}^2 = \sum_{k=1}^N e_{0,k} \frac{\partial e_{0,k}}{\partial y_{0,j}} = e_{0,j}.$$

El segundo factor es la derivada de la función de activación, y el tercer factor es

$$\frac{\partial v_{0,j}}{\partial w_{ji}^I} = \frac{\partial}{\partial w_{ji}^I} \sum_{\ell} w_{j\ell}^I x_{0,\ell} = x_{0,i}.$$

De esta forma nos queda

$$\frac{\partial E_0}{\partial w_{ji}^I} = (y_{0,j} - d_{0,j})(1 - y_{0,j})(1 + y_{0,j}) x_{0,i}.$$

Usando nuevamente la definición de $\delta_{0,j} \triangleq \frac{\partial E_0}{\partial y_{0,j}} \frac{\partial y_{0,j}}{\partial v_{0,j}}$, escribimos de forma simplificada

$$\frac{\partial E_0}{\partial w_{ji}^I} = \delta_{0,j} x_{0,i}.$$

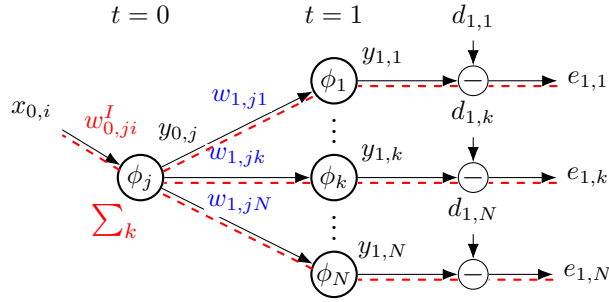
Ahora avanzamos a $t = 1$ y tenemos que considerar nuevamente los dos aportes: el directo en $t = 1$ y el indirecto en $t = 0$. Esto es,

$$\frac{\partial E_1}{\partial w_{ji}^I} = \frac{\partial E_1}{\partial w_{1,ji}^I} + \frac{\partial E_1}{\partial w_{0,ji}^I}.$$

Para el primer término sabemos que

$$\begin{aligned}
\frac{\partial E_1}{\partial w_{1,ji}^I} &= \frac{\partial E_1}{\partial y_{1,j}} \frac{\partial y_{1,j}}{\partial v_{1,j}} \frac{\partial v_{1,j}}{\partial w_{1,ji}^I} \\
&= e_{1,j} \cdot (1 - y_{1,j})(1 + y_{1,j}) \cdot x_{1,i} \\
&= \delta_{1,j} x_{1,i}
\end{aligned}$$

Para segundo término volvemos a usar j para hacer referencia a la versión de la neurona que está actuando en el tiempo $t = 0$, y recibe las entradas $x_{0,i}$ (también del tiempo $t = 0$), pero su salida alimenta a todas las neuronas k que están actuando en $t = 1$. El siguiente gráfico muestra las contribuciones en este caso.



Entonces, para el segundo término tenemos que

$$\begin{aligned}
\frac{\partial E_1}{\partial w_{0,ji}^I} &= \frac{\partial}{\partial y_{0,j}} \frac{1}{2} \sum_{k=1}^N e_{1,k}^2 \cdot (1 - y_{0,i})(1 + y_{0,i}) \cdot x_{0,i} \\
&= \sum_{k=1}^N e_{1,k} \frac{\partial e_{1,k}}{\partial y_{0,j}} \cdot (1 - y_{0,i})(1 + y_{0,i}) x_{0,i} \\
&= \sum_k e_{1,k} \cdot 1 \cdot (1 - y_{1,k})(1 + y_{1,k}) \cdot w_{1,jk} \cdot (1 - y_{0,i})(1 + y_{0,i}) x_{0,i} \\
&= \left(\sum_k \delta_{1,k} w_{jk} \right) (1 - y_{0,i})(1 + y_{0,i}) x_{0,i} \\
&= \delta_{0,j} x_{0,i}.
\end{aligned}$$

Luego, solo queda sumar los aportes directo e indirecto, $\partial E_1 / \partial w_{ji}^I = \delta_{1,j} x_{1,i} + \delta_{0,j} x_{0,i}$, que generalizando resulta en

$$\frac{\partial E_t}{\partial w_{ji}^I} = \sum_{\tau=t}^0 \delta_{\tau,j} x_{\tau,i},$$

con la misma definición recursiva que ya teníamos para los $\delta_{\tau,j}$. Finalmente, el gradiente global se obtiene sumando sobre todos los tiempos t hasta T .

El algoritmo de retropropagación a través del tiempo se puede resumir como muestra el Algoritmo 3.

Algoritmo 3 BPTT pasada hacia atrás (backward)

```
1: Entradas: Secuencia  $X$  (en one-hot), predicciones  $\mathbf{y}$ ,  
2:           salidas deseadas  $\mathbf{d}$  (secuencia corrida 1 posición)  
3:  $dW^I \leftarrow 0, dW \leftarrow 0$   
4: for  $t = 0$  to  $T - 1$  do  
5:    $\delta \leftarrow (\mathbf{y}_t - \mathbf{d}_t)$   
6:   for  $\tau = t$  to  $0$  do  
7:      $\delta \leftarrow \delta \odot (1 - \mathbf{y}_\tau) \odot (1 + \mathbf{y}_\tau)$   
8:      $dW^I \leftarrow dW^I + \delta \cdot \mathbf{x}_\tau^T$   
9:      $dW \leftarrow dW + \delta \cdot \mathbf{y}_{\tau-1}^T$   
10:     $\delta \leftarrow W^T \cdot \delta$   
11:   end for  
12: end for  
13: return  $dW^I, dW$ 
```

Este algoritmo es una variante del denominado BPTT de tiempo real, ya que calcula los gradientes luego de procesar cada elemento de la secuencia. Sin embargo, en lugar de actualizar los pesos en cada tiempo, se acumulan los gradientes a lo largo de toda la secuencia y se actualizan al final. Esto es útil para secuencias relativamente cortas, pero para secuencias largas puede ser muy costoso computacionalmente, ya que el número de operaciones crece con el cuadrado de la longitud de la secuencia T . Una versión comúnmente utilizada es la del BPTT truncada, que limita la cantidad de pasos hacia atrás que se consideran para el cálculo de los gradientes. Esto se puede implementar simplemente limitando el bucle **for** interno a un número fijo de pasos P , es decir: **for** $\tau = t$ **to** $\max(0, t - P)$.

3. BPTT optimizado

Si ahora pasamos a $t = 2$, la sumatoria de contribuciones de gradientes se extiende sobre

$$\frac{\partial E_2}{\partial w_{ji}} = \frac{\partial E_2}{\partial w_{2,ji}} + \frac{\partial E_2}{\partial w_{1,ji}} + \frac{\partial E_2}{\partial w_{0,ji}}.$$

Expandiendo un poco más cada término y simplificando la notación de los $w_{2,ji}$, $w_{1,ji}$ y $w_{0,ji}$, que son el mismo peso w_{ji} , podemos escribirlos usando la regla de la cadena y una notación simplificada como

$$\frac{\partial E_2}{\partial w} = \frac{\partial E_2}{\partial v_2} \frac{\partial v_2}{\partial w} + \frac{\partial E_2}{\partial v_2} \frac{\partial v_2}{\partial v_1} \frac{\partial v_1}{\partial w} + \frac{\partial E_2}{\partial v_2} \frac{\partial v_2}{\partial v_1} \frac{\partial v_1}{\partial v_0} \frac{\partial v_0}{\partial w}.$$

Además, también teníamos los correspondientes para E_0 y E_1 , que podemos escribir con la notación simplificada como

$$\frac{\partial E_0}{\partial w} = \frac{\partial E_0}{\partial v_0} \frac{\partial v_0}{\partial w},$$

y

$$\frac{\partial E_1}{\partial w} = \frac{\partial E_1}{\partial v_1} \frac{\partial v_1}{\partial w} + \frac{\partial E_1}{\partial v_1} \frac{\partial v_1}{\partial v_0} \frac{\partial v_0}{\partial w}.$$

Ahora, sumando todos los aportes y agrupando factores comunes,

$$\begin{aligned} \frac{\partial E}{\partial w} &= \frac{\partial E_0}{\partial w} + \frac{\partial E_1}{\partial w} + \frac{\partial E_2}{\partial w} \\ &= \frac{\partial E_0}{\partial v_0} \frac{\partial v_0}{\partial w} + \\ &\quad \frac{\partial E_1}{\partial v_1} \frac{\partial v_1}{\partial w} + \frac{\partial E_1}{\partial v_1} \frac{\partial v_1}{\partial v_0} \frac{\partial v_0}{\partial w} + \\ &\quad \frac{\partial E_2}{\partial v_2} \frac{\partial v_2}{\partial w} + \frac{\partial E_2}{\partial v_2} \frac{\partial v_2}{\partial v_1} \frac{\partial v_1}{\partial w} + \frac{\partial E_2}{\partial v_2} \frac{\partial v_2}{\partial v_1} \frac{\partial v_1}{\partial v_0} \frac{\partial v_0}{\partial w} \\ &= \left(\frac{\partial E_2}{\partial v_2} \right) \frac{\partial v_2}{\partial w} + \\ &\quad \left(\frac{\partial E_1}{\partial v_1} + \frac{\partial E_2}{\partial v_2} \frac{\partial v_2}{\partial v_1} \right) \frac{\partial v_1}{\partial w} + \\ &\quad \left(\frac{\partial E_0}{\partial v_0} + \left(\frac{\partial E_1}{\partial v_1} + \frac{\partial E_2}{\partial v_2} \frac{\partial v_2}{\partial v_1} \right) \frac{\partial v_1}{\partial v_0} \right) \frac{\partial v_0}{\partial w}. \end{aligned}$$

En la última igualdad, podemos interpretar que el $\left(\frac{\partial E_2}{\partial v_2} \right)$ que aparece en el primer término, se actualiza en el segundo término multiplicando por $\frac{\partial v_2}{\partial v_1}$ y sumando adelante $\frac{\partial E_1}{\partial v_1}$. Luego, en el tercer término, ese factor actualizado $\left(\frac{\partial E_1}{\partial v_1} + \frac{\partial E_2}{\partial v_2} \frac{\partial v_2}{\partial v_1} \right)$ se actualiza nuevamente multiplicando por $\frac{\partial v_1}{\partial v_0}$ y sumándole $\frac{\partial E_0}{\partial v_0}$.

Si llamamos δ^* a estos factores que se van actualizando, cuando estamos en $t = 2$ podemos escribirlo como

$$\delta_2^* = \frac{\partial E_2}{\partial v_2}.$$

Luego, en $t = 1$ nos queda

$$\delta_1^* = \frac{\partial E_1}{\partial v_1} + \delta_2^* \frac{\partial v_2}{\partial v_1},$$

y en $t = 0$,

$$\delta_0^* = \frac{\partial E_0}{\partial v_0} + \delta_1^* \frac{\partial v_1}{\partial v_0}.$$

Así, se puede escribir el gradiente como

$$\frac{\partial E}{\partial w} = \delta_2^* \frac{\partial v_2}{\partial w} + \delta_1^* \frac{\partial v_1}{\partial w} + \delta_0^* \frac{\partial v_0}{\partial w}.$$

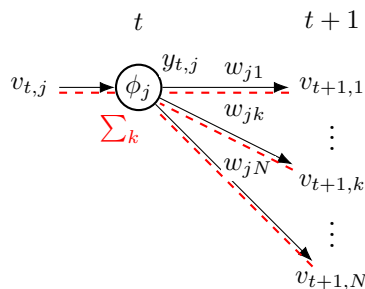
Generalizando, se puede definir el δ^* acumulativo de forma recursiva para cualquier t ,

$$\delta_t^* \triangleq \frac{\partial E_t}{\partial v_t} + \delta_{t+1}^* \frac{\partial v_{t+1}}{\partial v_t},$$

donde el primer término es el aporte directo del error en t , es decir

$$\frac{\partial E_t}{\partial v_t} = (y_t - d_t)(1 - y_t)(1 + y_t).$$

En cambio, en el segundo término en realidad hay que considerar, para cada neurona j , todos los aportes desde las neuronas k en $t + 1$, como ilustra la siguiente figura.



Así que, ahora usando la notación completa, tenemos

$$\sum_k \delta_{t+1,k}^* \frac{\partial v_{t+1,k}}{\partial v_{t,j}} = \sum_k \delta_{t+1,k}^* \frac{\partial v_{t+1,k}}{\partial y_{t,j}} \frac{\partial y_{t,j}}{\partial v_{t,j}} = \sum_k \delta_{t+1,k}^* w_{jk} (1 - y_{t,j})(1 + y_{t,j}).$$

Como se puede ver, en las dos últimas ecuaciones está el factor $(1 - y_{t,j})(1 + y_{t,j})$, por lo que volviendo a la definición del δ_t^* acumulativo y sacando ese factor común, se puede escribir de forma simplificada como

$$\delta_{t,j}^* = \left((y_{t,j} - d_{t,j}) + \sum_k w_{jk} \delta_{t+1,k}^* \right) (1 - y_{t,j})(1 + y_{t,j}).$$

Habiendo obtenido esta forma general para el δ^* acumulativo, podemos escribir el gradiente para cualquier t como

$$\frac{\partial E_t}{\partial w_{ji}} = \delta_{t,j}^* \frac{\partial v_{t,j}}{\partial w_{ji}},$$

que sumando sobre todos los t da el gradiente global

$$\frac{\partial E}{\partial w_{ji}} = \sum_{t=T-1}^0 \delta_{t,j}^* \frac{\partial v_{t,j}}{\partial w_{ji}}.$$

Finalmente, solo queda por reemplazar $\partial v_{t,j} / \partial w_{ji}$, que como ya sabemos es $y_{t-1,i}$, por lo que el gradiente para los pesos de las conexiones recurrentes queda

$$\frac{\partial E}{\partial w_{ji}} = \sum_{t=T-1}^0 \delta_{t,j}^* y_{t-1,i}.$$

De forma similar, para los pesos de las conexiones de entrada sabemos que $\partial v_{t,j}/\partial w_{ji}^I = x_{t,i}$, por lo que el gradiente queda

$$\frac{\partial E}{\partial w_{ji}^I} = \sum_{t=T-1}^0 \delta_{t,j}^* x_{t,i},$$

Esto permite evitar la sumatoria interna sobre τ que teníamos en el algoritmo BPTT original, reduciendo la complejidad computacional de $\mathcal{O}(T^2)$ a $\mathcal{O}(T)$. El Algoritmo 4 muestra una implementación de esta versión optimizada.

Algoritmo 4 BPTT optimizado

```

1: Entradas: Secuencia  $X$  (en one-hot), predicciones  $\mathbf{y}$ ,
2:           salidas deseadas  $\mathbf{d}$ 
3:  $dW^I \leftarrow 0, dW \leftarrow 0$ 
4:  $\delta^* \leftarrow 0$ 
5: for  $t = T - 1$  to  $0$  do
6:    $\delta^* \leftarrow (\mathbf{y}_t - \mathbf{d}_t) + \delta^*$ 
7:    $\delta^* \leftarrow \delta^* \odot (1 - \mathbf{y}_t) \odot (1 + \mathbf{y}_t)$ 
8:    $dW^I \leftarrow dW^I + \delta^* \cdot \mathbf{x}_t^T$ 
9:    $dW \leftarrow dW + \delta^* \cdot \mathbf{y}_{t-1}^T$ 
10:   $\delta^* \leftarrow W^T \cdot \delta^*$ 
11: end for
12: return  $dW^I, dW$ 

```
